Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_2**	
Experiment:-8	**Date:21-08-2026**	**Enrollment No:92400133057**

GIT HUB:- https://github.com/manavkasundra/DSIP

Aim:-image processing_2

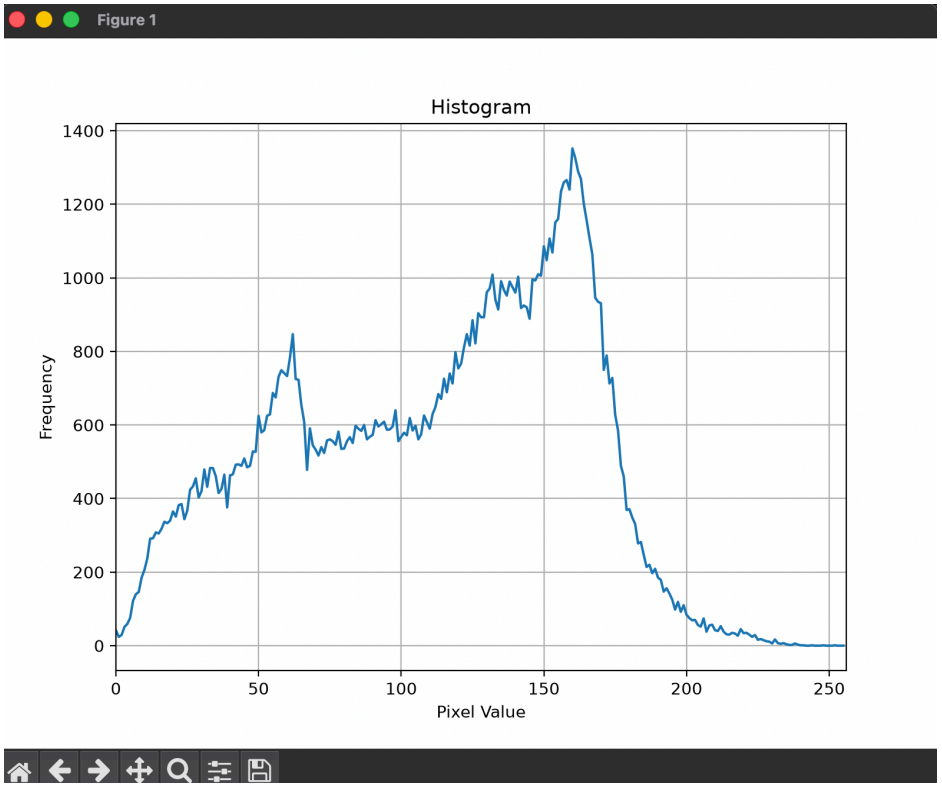
Code:-

```
import cv2
import matplotlib.pyplot as plt
# Load the image
image_path = "img.jpg"  # Replace with the path to your image
image = cv2.imread(image_path, cv2.IMREAD_GRAYSCALE)
# Calculate the histogram
histogram = cv2.calcHist([image], [0], None, [256], [0, 256])
# Plot the histogram
plt.figure(figsize=(8, 6))
plt.title('Histogram')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')
plt.plot(histogram)
plt.xlim([0, 256])
plt.grid(True)
plt.show()
# Perform histogram equalization
equalized_image = cv2.equalizeHist(image)
# Display the original and equalized images
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.title('Original Image')
plt.imshow(image, cmap='gray')
plt.axis('off')
plt.subplot(1, 2, 2)
plt.title('Equalized Image')
plt.imshow(equalized_image, cmap='gray')
plt.axis('off')
# Calculate the histogram
histogram = cv2.calcHist([equalized_image], [0], None, [256], [0, 256])
```

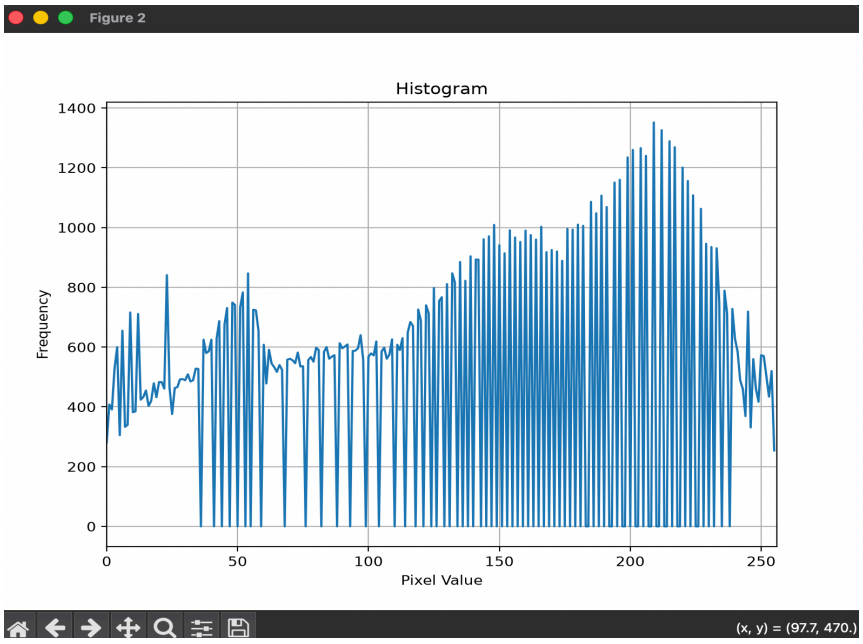
Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_2**	
Experiment:-8	**Date:21-08-2026**	**Enrollment No:92400133057**

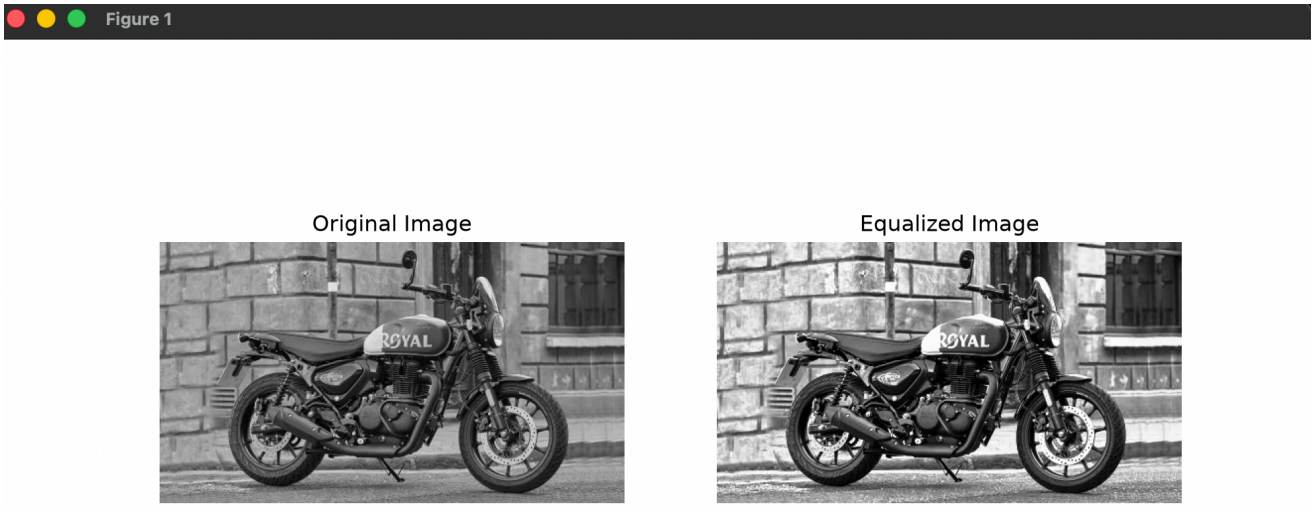
```
# Plot the histogram
plt.figure(figsize=(8, 6))
plt.title('Histogram')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')
plt.plot(histogram)
plt.xlim([0, 256])
plt.grid(True)
plt.show()
plt.tight_layout()
plt.show()
```

output:-



Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_2**	
Experiment:-8	**Date:21-08-2026**	**Enrollment No:92400133057**





Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_2**	
Experiment:-8	**Date:21-08-2026**	**Enrollment No:92400133057**

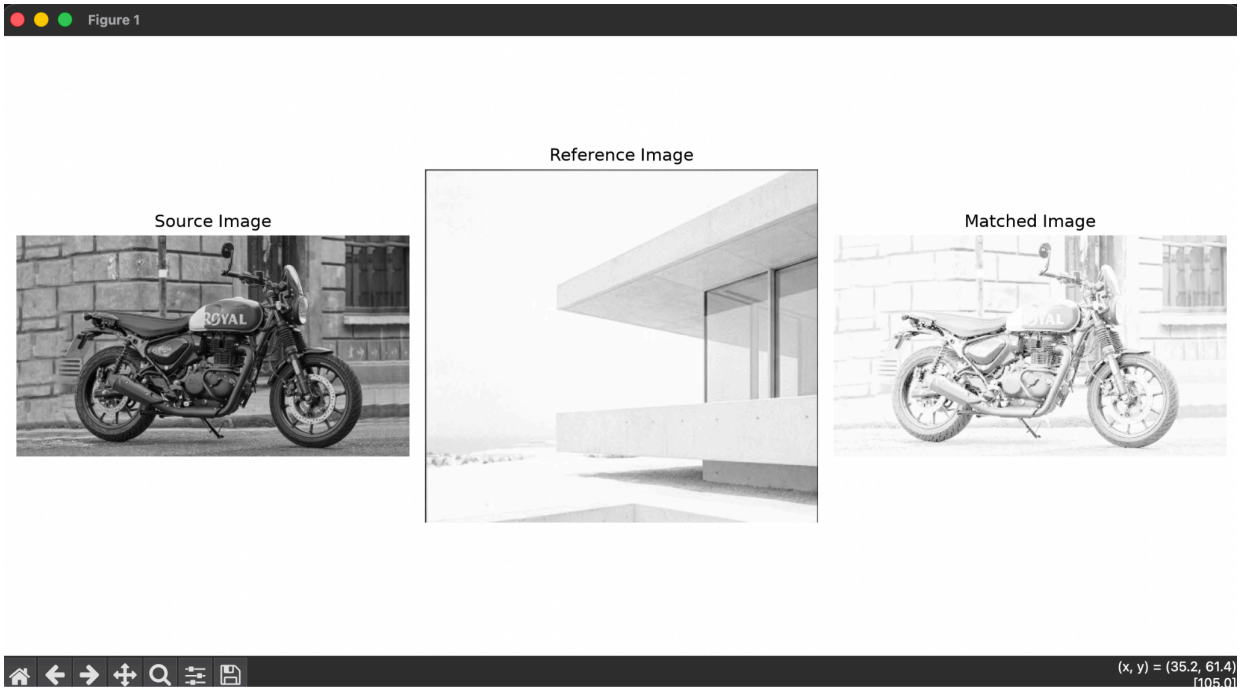
Code2:-

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
# Load the source and reference images
source_path = 'img.jpg'
reference_path = 'img_1.png'
source_image = cv2.imread(source_path, cv2.IMREAD_GRAYSCALE)
reference_image = cv2.imread(reference_path, cv2.IMREAD_GRAYSCALE)
# Calculate histograms for the source and reference images
source_hist = cv2.calcHist([source_image], [0], None, [256], [0, 256])
reference_hist = cv2.calcHist([reference_image], [0], None, [256], [0, 256])
# Normalize histograms to have sum equal to 1
source_hist /= source_hist.sum()
reference_hist /= reference_hist.sum()
# Calculate cumulative distribution functions (CDF) for histograms
source_cdf = source_hist.cumsum()
reference_cdf = reference_hist.cumsum()
# Perform histogram matching by mapping source CDF to reference CDF
mapping = np.interp(source_cdf, reference_cdf, range(256))
matched_image = mapping[source_image]
# Convert to uint8 data type
matched_image = matched_image.astype(np.uint8)
# Display the images using Matplotlib
plt.figure(figsize=(12, 6))
plt.subplot(131)
plt.title('Source Image')
plt.imshow(source_image, cmap='gray')
plt.axis('off')
plt.subplot(132)
plt.title('Reference Image')
plt.imshow(reference_image, cmap='gray')
plt.axis('off')
plt.subplot(133)
```

Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_2**	
Experiment:-8	**Date:21-08-2026**	**Enrollment No:92400133057**

```
plt.title('Matched Image')
plt.imshow(matched_image, cmap='gray')
plt.axis('off')
plt.tight_layout()
plt.show()
```

output:-



Marwadi University Marwadi Chandarana Group NAAC A+	**Marwadi University** **Faculty of Engineering &Technology**	
Subject: DSIP	**Aim: Image processing_2**	
Experiment:-8	**Date:21-08-2026**	**Enrollment No:92400133057**

Conclusion:-

This experiment effectively demonstrates the core principles of histogram equalization and histogram matching in digital image processing. Histogram equalization optimizes image contrast by flattening and expanding the intensity profile across available gray levels, yielding clearer detail and enhanced visual balance. Concurrently, histogram matching (specification) enables precise tonal alignment by mapping the source image's intensity profile to match a designated target distribution, making it ideal for visual consistency and stylistic adjustments. Overall, both methods underscore how reshaping pixel distributions serves as a robust approach for targeted image enhancement and standardization.